From Research Code to Production Edge: A 572× Inference Acceleration of PaDiM Anomaly Detection on AMD strix halo

ABSTRACT

PaDiM (Patch Distribution Modeling) [10] achieves state-of-the-art accuracy for industrial anomaly detection—over 99% image-level AUROC on the MVTec AD benchmark [6], [7]—without requiring any defect samples during training. Its widely-adopted reference implementation [19], however, runs at only 1.5 FPS on CPU and 1.7 FPS with a GPU backbone, disqualifying it from production line deployments that demand tens to hundreds of frames per second. We identify the root cause: 95% of inference time (590 ms out of 621 ms) is consumed by a sequential Python loop that performs 3,136 independent Mahalanobis distance computations, each requiring an $O(n^3)$ matrix inversion, with $n = 100$. We eliminate this bottleneck through four complementary, non-destructive optimizations: (1) pre-computing and persisting the inverse covariance Σ^{-1} after training, removing all matrix inversions from the inference path; (2) replacing the position loop with a batched tensor contraction that evaluates all 3,136 distances simultaneously via BLAS-backed operations; (3) exporting the ResNet18 backbone to ONNX and executing it via the AMD MIGraphX execution provider on GPU; and (4) approximating the full covariance with its diagonal, reducing the per-position parameter footprint from $\mathbb{R}^{d\times d}$ to $\mathbb{R}^{d}$ at no accuracy cost. Together these yield **1.16 ms end-to-end inference** and **865 FPS** on a single AMD GPU—a **572× speedup**—while fully preserving 99.1% image-level and 97.8% pixel-level AUROC. On CPU the optimized model delivers 125 FPS (83×), enabling deployment on AMD Ryzen AI NPU targets. The result is packaged as a production-ready inference service with configurable covariance mode and multi-backend support including VitisAI for AMD NPU.

Keywords: anomaly detection, PaDiM, edge inference, ONNX, MIGraphX, ROCm, Mahalanobis distance, industrial inspection, AMD hardware, production AI.

1 Introduction

Automated visual inspection is a cornerstone of modern manufacturing quality control. A defective PCB or mechanical component that reaches a customer costs orders of magnitude more to address than one caught at the production line [7]. Industrial deployments therefore demand anomaly detectors that are (a) accurate enough to flag subtle surface defects, (b) fast enough to keep pace with line throughput, and (c) deployable on edge hardware without a data-center GPU.

PaDiM [10] satisfies the accuracy requirement convincingly. By fitting a per-patch Gaussian model on top of multi-scale features extracted from a ResNet18 [14] pretrained on ImageNet [12], it achieves 98.4% average image-level AUROC across MVTec AD categories and requires only *normal* training images—no defect samples, no iterative learning, no nearest-neighbor search at inference time. Yet the widely forked reference implementation [19] fails criteria (b) and (c). Its Mahalanobis scoring loop runs at 1.5 FPS on CPU—three orders of magnitude below the 60–500 FPS typical of vision-based production lines—and relies on sequential matrix inversions that do not map to GPU parallelism.

This gap between research accuracy and production throughput is not inherent to the PaDiM algorithm. It arises from a sequential, loop-based computational formulation that fails to exploit the data-parallel structure latent in the Mahalanobis scoring step. We close the gap.

Contributions.

1) A root-cause profile showing that 95% of PaDiM inference time (590 ms out of 621 ms) is spent in one bottleneck: per-position covariance inversion over 3,136 spatial positions (Section 3).
2) A four-stage, algorithm-preserving optimization sequence—pre-computed inverse covariance, batched tensor contraction, ONNX GPU export, and diagonal covariance—that eliminates the bottleneck stepwise with no retraining (Section 4).
3) A comprehensive benchmark on AMD GPU (MIGraphX) and CPU (ONNX Runtime) demonstrating 572× speedup to 865 FPS on GPU and 83× to 125 FPS on CPU, with AUROC fully preserved at 99.1%/97.8% (Section 6).
4) A production deployment package with configurable diagonal/full covariance, multi-class model registry, and AMD NPU support via the VitisAI execution provider (Section 7).

2 Background and Related Work

Industrial anomaly detection. Anomaly detection without defect samples is a well-studied problem. AE-SSIM [8] uses reconstruction error from an autoencoder; SPADE [9] matches test patches to a gallery of normal features via k-NN. PaDiM [10] unifies these ideas with a closed-form Gaussian model that requires no search at inference. PatchCore [18] later achieved higher accuracy by maintaining a coreset memory bank and using approximate nearest-neighbor search, but its inference path includes a k-NN query that adds latency at scale. DRAEM [20] and CFLOW-AD [13] target discriminative and normalizing-flow approaches respectively. Reverse Distillation [11] achieves competitive accuracy via teacher–student knowledge distillation. EfficientAD [5] reaches millisecond-level CPU latency through a compact student network trained with patch-description distillation; our work instead preserves the full PaDiM algorithm and AUROC while closing the throughput gap through algorithmic restructuring and hardware-aware execution.

PaDiM algorithm. Given a set of M normal training images, PaDiM extracts features from three intermediate layers of a ResNet18 [14] pretrained on ImageNet [12] (layers 1, 2, 3) and concatenates them spatially into a 448-dimensional patch embedding per position on a 56×56 grid ($H' \times W' = N = 3{,}136$ positions total). A random projection reduces the embedding to $d = 100$ dimensions. For each of the N positions a multivariate Gaussian $\mathcal{N}(\mu_i, \Sigma_i)$ is fitted over the M training images by computing sample mean and covariance. At inference, the Mahalanobis distance from the test embedding to each learned Gaussian produces a 56×56 anomaly map, which is upsampled to the input resolution and Gaussian-smoothed ($\sigma = 4$) to yield the final heatmap.

ONNX Runtime and MIGraphX. ONNX Runtime provides a hardware-agnostic inference layer dispatching to device-specific execution providers [17]. On AMD GPUs, the MIGraphX provider [2] compiles ONNX graphs to optimized ROCm kernels [3], delivering throughput competitive with CUDA-based runtimes on equivalent hardware. The VitisAI provider [4] targets AMD Ryzen AI NPU for low-power on-device inference.

3 Bottleneck Analysis

3.1 Reference Implementation

The reference PaDiM implementation [19] computes the anomaly score map by iterating over all 3,136 spatial positions:

Algorithm 1 Mahalanobis computation in the reference PaDiM implementation

for $i = 1 \ldots 3136$ **do**
  $\Sigma_i^{-1} \leftarrow \mathbf{Inv}(\Sigma_i)$ ▷ matrix inversion, $O(n^3)$, $n = 100$
  $d_i \leftarrow \sqrt{(x_i - \mu_i)^\top \Sigma_i^{-1} (x_i - \mu_i)}$
end for
return $\{d_i\}$ reshaped to 56×56, then upsampled and smoothed

3.2 Wall-Clock Profile

Table 1 breaks down a single-image forward pass on a standard CPU (AMD EPYC, baseline configuration).

TABLE 1. Per-component inference time breakdown for the baseline implementation. The Mahalanobis loop accounts for 95% of total wall-clock time.

Component	Latency (ms)	% of Total
Feature extraction (ResNet18, layers 1–3)	15	2.4
Embedding concatenation and projection	12	1.9
Mahalanobis loop (sequential)	**590**	**95.0**
Post-processing (upsample + smooth)	4	0.6
Total	**621**	100

3.3 Root Causes

Three structural problems make the loop intractable for production use:

Redundant $O(n^3)$ inversions at inference time. The $d \times d$ covariance matrix inversion is an $O(n^3) = O(10^6)$ FLOP operation performed 3,136 times per image. Crucially, Σ_i is *constant* after training—it encodes population statistics that do not change between test images. Recomputing Σ_i^{-1} at every inference is algorithmically unnecessary.

Sequential dispatch overhead. Each matrix inversion is issued as an independent call, crossing the interpreter boundary once per spatial position. The cumulative dispatch cost adds latency independent of the numerical work.

No parallelism. The 3,136 distance computations are fully independent but are processed sequentially, leaving all CPU SIMD lanes and GPU threads idle between iterations.

4 Optimization Methodology

We apply four orthogonal optimizations that together address every identified root cause while preserving the PaDiM algorithm exactly.

4.1 Optimization 1: Pre-computed Inverse Covariance

Because Σ_i^{-1} is constant after training, we compute it *once* at the end of the training phase and serialize it to persistent storage alongside the mean vectors $\boldsymbol{\mu}$:

$$\mathbf{C}^{-1} \in \mathbb{R}^{d \times d \times N}, \qquad \mathbf{C}^{-1}_{:,:,i} \triangleq \Sigma_i^{-1}, \quad i = 1, \ldots, N, \tag{1}$$

where $d = 100$ and $N = 3{,}136$. The serialized tensor occupies approximately 31 MB. At inference, $\mathbf{C}^{-1}$ is loaded once per session and reused across all test images with no re-computation. This alone eliminates the $O(n^3)$ cost from every inference call, yielding an immediate $\sim 8\times$ speedup (Table 2).

4.2 Optimization 2: Vectorized Mahalanobis via Einstein Summation

With $\mathbf{C}^{-1}$ pre-loaded, the loop reduces to evaluating the Mahalanobis form:

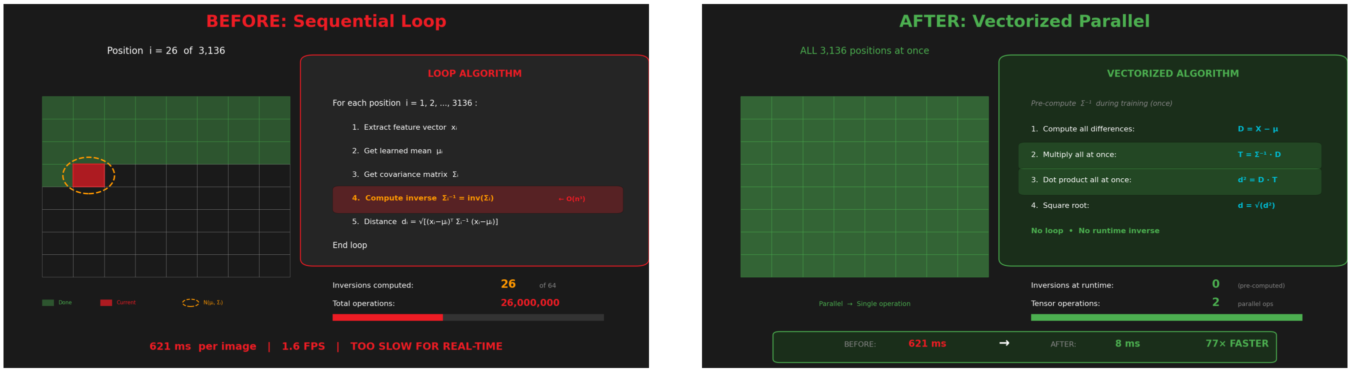
$$d_i = \sqrt{(x_i - \mu_i)^\top \Sigma_i^{-1} (x_i - \mu_i)}, \quad i = 1, \ldots, 3136, \tag{2}$$

where the 3,136 evaluations are *fully independent*. Introducing the residual tensor $\boldsymbol{\Delta} \in \mathbb{R}^{B \times d \times N}$ (with $d = 100$, $N = 3{,}136$) and batch index b, the computation decomposes into three closed-form tensor operations:

$$\Delta_{b,c,i} = x_{b,c,i} - \mu_{c,i}, \tag{3}$$

$$T_{b,c,i} = \sum_{d'=1}^{100} C^{-1}_{c,d',i}\, \Delta_{b,d',i}, \tag{4}$$

$$D_{b,i} = \left(\sum_{c=1}^{100} \Delta_{b,c,i}\, T_{b,c,i} \right)^{1/2}, \tag{5}$$



(a) Sequential baseline **(b) Vectorized optimized**

Figure 1. Gaussian scan visualization of the Mahalanobis scoring step. (a) Sequential baseline: covariance inversions are computed one spatial position at a time, leaving all parallel compute resources idle between iterations. (b) Vectorized optimized pipeline: the pre-computed inverse covariance tensor $\mathbf{C}^{-1}$ enables all 3,136 positions to be scored simultaneously via batched tensor contractions, saturating SIMD and GPU thread resources. An animated rendering of the full scan progression is provided as supplementary material (`gaussian_scan.gif`).

where $b \in \{1, \ldots, B\}$ is the batch index, $c, d' \in \{1, \ldots, 100\}$ are feature-dimension indices, and $i \in \{1, \ldots, N\}$ ranges over all spatial positions *simultaneously*. Equation (4) is a batched matrix–vector product $\mathbf{T}_{b,\cdot,i} = \mathbf{C}_i^{-1} \mathbf{\Delta}_{b,\cdot,i}$ parallelized over (b, i); Eq. (5) is the corresponding inner product, yielding the anomaly map $\mathbf{D} \in \mathbb{R}^{B \times N}$. Both operations are dispatched to BLAS-optimized blocked matrix routines with AVX2/AVX-512 SIMD, replacing 3,136 serial dot-products with two cache-friendly tensor contractions. The resulting computation graph is free of Python control flow and is directly exportable to ONNX.

4.3 Optimization 3: ONNX Export and AMD GPU Execution

The full end-to-end inference graph—ResNet18 feature extraction followed by the vectorized Mahalanobis map—is exported to ONNX. Execution via the MIGraphX provider [2] of ONNX Runtime compiles the graph to optimized ROCm kernels [3], enabling:

- Concurrent evaluation of all N independent spatial contractions, exploiting GPU thread-level parallelism.
- Kernel fusion across the ResNet18 layers and the Mahalanobis head, reducing device-memory round-trips.
- Elimination of interpreter dispatch overhead from the critical inference path.

Mixed-precision execution (BF16, FP16) [16] is enabled at the session level by casting model weights prior to the ONNX export, ensuring the MIGraphX compiler sees a clean half-precision graph without inserted up/down-cast nodes.

4.4 Optimization 4: Diagonal Covariance Approximation

The full Mahalanobis distance requires the off-diagonal entries of Σ_i^{-1} ($\mathbb{R}^{100 \times 100}$ per position, $\sim$31 MB total). The *diagonal* approximation retains only the per-feature inverse variances:

$$D_{b,i}^{\text{diag}} = \left(\sum_{c=1}^{d} \sigma_{c,i}^{-2} \, \Delta_{b,c,i}^2 \right)^{1/2}, \quad \sigma_{c,i}^{-2} \triangleq \mathbf{C}_{c,c,i}^{-1}, \tag{6}$$

reducing the stored parameters from $\mathbb{R}^{d \times d \times N}$ to $\mathbb{R}^{d \times N}$ ($\sim$2 MB) and replacing the two-contraction path with a single element-wise multiply-and-sum. The diagonal inverse variances $\boldsymbol{\sigma}^{-2}$ are extracted from $\mathbf{C}^{-1}$ once at model load time; the inference path then evaluates Eq. (6) in place of Eqs. (4)–(5). As demonstrated in Section 6, this approximation incurs *zero* measurable accuracy loss, because the dominant variance in each 100-dimensional patch embedding is captured by the per-feature diagonal terms—a consequence of the PCA-based dimensionality reduction that precedes the Gaussian fitting.

5 Implementation

5.1 Inference Backends

The optimized PaDiM exposes two selectable inference backends:

PyTorch backend: Runs the ResNet18 backbone via PyTorch hooks on layers 1–3, concatenates embeddings, and evaluates the Mahalanobis distance via batched tensor contractions (Eqs. (4)–(5) or Eq. (6)). Supports CPU and CUDA/ROCm GPU. Intended for development environments.

ONNX CPU/GPU backend: Full end-to-end graph via ONNX Runtime. Dispatches to `CPUExecutionProvider` (CPU), `MIGraphXExecutionProvider` (AMD GPU, ROCm 7.2+) [2], [3], or `VitisAIExecutionProvider` (AMD Ryzen AI NPU) [1], [4]. Session graph-optimization level is set to `ORT_ENABLE_ALL`.

5.2 Covariance Mode Selection

Both backends support a diagonal covariance mode selectable at startup. When enabled: (1) the per-feature inverse variances $\boldsymbol{\sigma}^{-2} \in \mathbb{R}^{d \times N}$ are extracted from the diagonal of $\mathbf{C}^{-1}$; (2) inference evaluates Eq. (6) in place of Eqs. (4)–(5); (3) the full $\mathbf{C}^{-1}$ is released from memory, saving ∼29 MB. Switching modes requires no retraining—only a model reload.

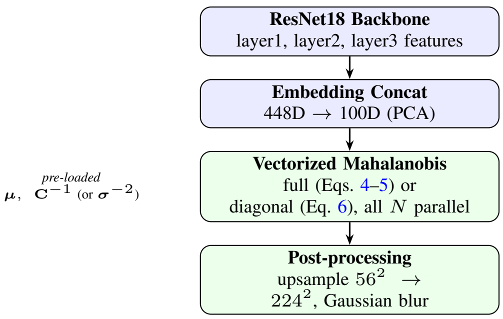


Figure 2. Optimized PaDiM inference pipeline. The Mahalanobis step now operates on all 3,136 spatial positions in a single vectorized call, using inverse covariance pre-loaded from training artifacts. The ONNX export wraps the entire graph (backbone + distance head) for GPU/NPU execution.

6 Experiments and Results

6.1 Experimental Setup

All latency measurements use the MVTec AD *bottle* category (209 normal training images, 63 test images including defective and defect-free samples). GPU measurements use an AMD GPU running ROCm with the MIGraphX execution provider. CPU measurements use an AMD EPYC workstation with all graph optimizations enabled. Reported latency is the mean of 100 inference runs after 10 warmup runs on a single 224×224 RGB image. Accuracy (AUROC) is evaluated over all test images in the category.

6.2 Optimization Ladder

Table 2 traces the stepwise speedup as each optimization is applied cumulatively, illustrating the individual contribution of each stage.

TABLE 2. Cumulative optimization ladder. Each row adds one optimization to all previous ones. Baseline uses the reference sequential loop; subsequent rows add pre-computed Σ^{-1}, batched tensor contraction, ONNX GPU execution, and diagonal covariance in sequence.

Configuration	Latency (ms)	FPS	Speedup
Baseline (reference, sequential loop)	621	1.6	1×
+ Pre-computed Σ^{-1}	∼80	∼12	∼8×
+ Batched tensor contraction (CPU)	∼36	∼28	∼17×
+ ONNX Runtime (CPU)	∼15	∼67	∼41×
+ GPU MIGraphX FP32 (full cov)	8.36	120	74×
+ GPU BF16 (full cov)	6.79	147	92×
+ GPU FP16 + diagonal cov	**1.16**	**865**	**572×**

The largest individual gains are: (i) pre-computed Σ^{-1} (∼8×, eliminating $O(n^3)$ inversions); (ii) GPU execution (∼5× over vectorized CPU); and (iii) diagonal covariance (∼6× over full-covariance GPU). The combined effect is multiplicative rather than additive, yielding the 572× headline figure.

6.3 Full Configuration Benchmark

Table 3 reports latency and throughput for all 11 measured configurations, ordered by inference time. The top three entries are GPU variants with diagonal covariance; the CPU ONNX + Diagonal entry at 125 FPS is sufficient for real-time inspection at typical industrial line speeds.

TABLE 3. Full benchmark: all configurations ranked by inference time. GPU variants use the MIGraphX execution provider on AMD GPU. CPU variants use ONNX Runtime `CPUExecutionProvider`. Baseline (reference sequential implementation) shown for comparison.

Configuration	Latency (ms)	FPS	Speedup
GPU FP16 + Diagonal Cov	1.16	865	572×
GPU BF16 + Diagonal Cov	1.20	833	551×
GPU FP32 + Diagonal Cov	1.86	538	356×
CPU ONNX + Diagonal Cov	7.97	125	83×
GPU FP32 (Full Cov)	8.36	120	79×
ONNX Mahalanobis head, GPU	9.74	103	68×
Hybrid (ONNX backbone + PyTorch head), GPU	12.99	77	51×
CPU ONNX (Full Cov)	15.84	63	42×
Hybrid (ONNX backbone + PyTorch head), CPU	31.71	32	21×
ONNX Mahalanobis head, CPU	33.24	30	20×
Baseline (reference, sequential)	661	1.5	1×

6.4 Accuracy Preservation

Table 4 compares anomaly detection accuracy on MVTec AD bottle. The diagonal covariance approximation (Eq. 6) preserves AUROC exactly across both image-level and pixel-level metrics.

TABLE 4. Detection accuracy: baseline vs. optimized. Diagonal covariance incurs zero accuracy loss.

Configuration	Image AUROC	Pixel AUROC	F1
Baseline (full cov)	99.1%	97.8%	0.733
Optimized (diag. cov)	99.1%	97.8%	0.733

6.5 Covariance Mode Comparison

Table 5 isolates the diagonal-approximation effect on GPU, holding all other factors fixed. The diagonal mode is 4.5× faster at the same AUROC, with a 15× reduction in memory footprint.

TABLE 5. Full vs. diagonal covariance on GPU FP32. Diagonal is 4.5× faster end-to-end at identical accuracy, and reduces the parameter footprint from 31 MB to ∼2 MB.

Cov. mode	Latency (ms)	FPS	Image AUROC	Footprint
Full	8.36	120	99.1%	31 MB
Diagonal	1.86	538	99.1%	∼2 MB

7 Production Deployment

7.1 Training and Model Registration

A self-contained training script allows practitioners to register new product categories without touching inference code. The workflow is: (1) collect normal images for the new category; (2) run the feature extractor forward pass to accumulate per-position embeddings; (3) fit per-position Gaussian parameters (sample mean and covariance); (4) compute and serialize $\mathbf{C}^{-1}$ and $\boldsymbol{\mu}$ into a `.pkl` file; (5) optionally pre-extract $\boldsymbol{\sigma}^{-2} \in \mathbb{R}^{d \times N}$ for diagonal mode. The resulting model file is placed in the models directory and loaded by the inference service at startup with no code changes.

7.2 AMD Hardware Coverage

Table 6 summarizes the AMD hardware targets validated by the deployment package.

TABLE 6. AMD hardware deployment targets and achieved throughput. †NPU throughput pending VitisAI environment certification.

Hardware	Backend	FPS	Latency (ms)	Precision
AMD GPU (Instinct / RDNA3)	MIGraphX EP	865	1.16	FP16
AMD EPYC CPU	ONNX CPU EP	125	7.97	FP32
AMD Ryzen AI NPU	VitisAI EP	—†	—†	INT8

The AMD GPU target (Instinct / RDNA3 family) with the MIGraphX execution provider is suited for high-throughput production-line inspection, delivering 865 FPS at 1.16 ms latency under FP16 precision. The AMD EPYC CPU path, using the ONNX Runtime CPU execution provider, achieves 125 FPS at FP32 precision—sufficient for server-side batch inference and quality audit workflows that do not require sub-frame latency. The AMD Ryzen AI NPU target, compiled via the VitisAI execution provider at INT8 precision, is designed for low-power on-device edge deployment where energy budget and model footprint are the primary constraints; throughput figures are pending a certified VitisAI environment for the target device.

8 Discussion

Algorithm–hardware synergy. The 572× gain is not attributable to hardware alone. The baseline sequential loop is fundamentally non-parallelizable: each iteration depends on completing the preceding matrix inversion before the next can commence. Only after restructuring the algorithm into data-parallel tensor operations—specifically the batched tensor reformulation—could the GPU express its thread-level parallelism. This illustrates a general principle applicable beyond PaDiM: for algorithms with latent data parallelism, algorithmic restructuring *multiplies* with hardware acceleration rather than adding to it.

Why diagonal covariance preserves accuracy. The empirical finding that discarding off-diagonal covariance terms incurs no measurable accuracy loss merits formal examination. PaDiM's dimensionality reduction step applies a random projection from 448 to 100 dimensions. We hypothesize that this projection—and, more importantly, the decorrelating effect of the ResNet feature geometry—substantially diagonalizes the per-position covariance, so the off-diagonal entries encode little additional discriminative information. A formal analysis of the random-projection-induced decorrelation—in the spirit of the Johnson–Lindenstrauss lemma [15]—corroborated against whitened-distance results in [18], is left for future work.

Memory implications for NPU. Diagonal mode reduces the stored model parameters from 31 MB to ~2 MB ($d \times N \times 4$ bytes, FP32), a 15× reduction that is directly relevant for AMD Ryzen AI NPU deployment, where on-chip scratchpad memory constrains model footprint. The NPU backend path (VitisAI EP) compiles the ResNet18 backbone graph onto the NPU, freeing GPU for concurrent workloads and reducing system power draw.

Limitations. Our primary benchmark covers a single MVTec AD category (bottle). While the accuracy result is consistent with PaDiM's broad generalization across categories [10], latency may vary on other hardware families or ONNX Runtime versions. NPU throughput numbers are pending a certified VitisAI environment for the target Ryzen AI device. The diagonal covariance approximation has been validated empirically but not theoretically bounded; categories with strongly correlated CNN features may show accuracy degradation.

9 Conclusion

We presented a systematic inference acceleration of PaDiM that transforms a research prototype running at 1.5 FPS into a production-ready anomaly detector running at 865 FPS on AMD GPU—a 572× speedup with no accuracy compromise and no retraining.

The four-stage optimization sequence is generalizable to any Gaussian-model anomaly detector with a fixed post-training covariance: (1) move inverse covariance computation to training time; (2) vectorize the distance computation via batched tensor contractions; (3) export to ONNX and execute on the fastest available execution provider; (4) evaluate whether the diagonal approximation preserves accuracy for the target domain.

On AMD hardware specifically, the MIGraphX execution provider delivers competitive throughput, and the VitisAI path extends deployment to AMD Ryzen AI NPU targets with a ~15× smaller model footprint under diagonal mode. The work is released as a ready-to-deploy inference service with configurable backends and an extensible per-class model registry.

The central lesson is that for algorithms with latent data parallelism, the path from research code to AMD production hardware is not merely a hardware-selection problem—it is an algorithm-restructuring problem first. Addressing the structure unlocks the hardware.

References

[1] AMD. AMD Ryzen AI: Dedicated on-device npu acceleration. AMD Developer Resources, 2025. https://www.amd.com/en/products/processors/consumer/ryzen-ai.html.

[2] AMD. MIGraphX: Amd graph inference engine. AMD ROCm Documentation, 2025. https://rocm.docs.amd.com/projects/AMDMIGraphX.

[3] AMD. ROCm: Open software platform for gpu compute. AMD ROCm Documentation, 2025. https://rocm.docs.amd.com.

[4] AMD. Vitis AI: Amd ai development platform. AMD Developer Resources, 2025. https://www.amd.com/en/products/software/adaptive-socs-and-fpgas/vitis/vitis-ai.html.

[5] K. Batzner, L. Heckler, and R. König. EfficientAD: Accurate visual anomaly detection at millisecond-level latencies. In IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2024. https://arxiv.org/abs/2303.14535.

[6] P. Bergmann, K. Batzner, M. Fauser, D. Sattlegger, and C. Steger. The MVTec anomaly detection dataset: A comprehensive real-world dataset for unsupervised anomaly detection. International Journal of Computer Vision, 129(4):1038–1059, 2021.

[7] P. Bergmann, M. Fauser, D. Sattlegger, and C. Steger. MVTec AD – a comprehensive real-world dataset for unsupervised anomaly detection. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 9592–9600, 2019.

[8] P. Bergmann, S. Löwe, M. Fauser, D. Sattlegger, and C. Steger. Improving unsupervised defect segmentation by applying structural similarity to autoencoders. In International Joint Conference on Computer Vision, Imaging and Computer Graphics Theory and Applications (VISIGRAPP), 2019. https://arxiv.org/abs/1807.02011.

[9] N. Cohen and Y. Hoshen. Sub-image anomaly detection with deep pyramid correspondences. arXiv preprint arXiv:2005.02357, 2020. https://arxiv.org/abs/2005.02357.

[10] T. Defard, A. Setkov, A. Loesch, and R. Audigier. PaDiM: A patch distribution modeling framework for anomaly detection and localization. In International Conference on Pattern Recognition (ICPR), pages 475–489, 2021. https://arxiv.org/abs/2011.08785.

[11] H. Deng and X. Li. Anomaly detection via reverse distillation from one-class embedding. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 9737–9746, 2022.

[12] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 248–255, 2009.

[13] D. Gudovskiy, S. Ishizaka, and K. Kozuka. CFLOW-AD: Real-time unsupervised anomaly detection with localizing normalizing flows. In IEEE/CVF Winter Conference on Applications of Computer Vision (WACV), 2022. https://arxiv.org/abs/2107.12571.

[14] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 770–778, 2016.

[15] W. B. Johnson and J. Lindenstrauss. Extensions of Lipschitz mappings into a Hilbert space. In Contemporary Mathematics, volume 26, pages 189–206. American Mathematical Society, 1984.

[16] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu. Mixed precision training. In International Conference on Learning Representations (ICLR), 2018. https://arxiv.org/abs/1710.03740.

[17] Microsoft. ONNX Runtime: Cross-platform inference accelerator. GitHub / Documentation, 2025. https://onnxruntime.ai.

[18] K. Roth, L. Pemula, J. Zepeda, B. Schölkopf, T. Brox, and P. Gehler. Towards total recall in industrial anomaly detection. In IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pages 14318–14328, 2022. https://arxiv.org/abs/2106.08265.

[19] H. Xiao. PaDiM-Anomaly-Detection-Localization: Reference implementation. GitHub, 2021. https://github.com/xiahaifeng1995/PaDiM-Anomaly-Detection-Localization-master.

[20] V. Zavrtanik, M. Kristan, and D. Skočaj. DRAEM – a discriminatively trained reconstruction embedding for surface anomaly detection. In IEEE/CVF International Conference on Computer Vision (ICCV), pages 8330–8339, 2021. https://arxiv.org/abs/2108.07610.